Response to the comments given by the reviewers of Manuscript:

ASE 2026 #691, Major Revision

(It Takes Two to Tango: Accelerating Code Generation via Cache-Inspired Hierarchical Speculative Decoding)

Dear Reviewers,

We would like to thank you for the constructive comments and recommendations to improve the manuscript. We have addressed all the comments. The details of our response are introduced below. We hope our responses could effectively address the raised concerns.

Yours sincerely,

Authors of ASE Submission #691

[Meta-Review]#1: Making the soundness/losslessness argument explicit in the manuscript, showing how the Leviathan et al. verification/rejection procedure applies to the tree construction, Preliminary Prediction, and Draft Expansion, and temperature sampling settings.

Response: Thank you for the suggestion. We have revised the manuscript to make the soundness/losslessness argument explicit.

Specifically, we added a detailed explanation in Section 2.1 of how the verification/rejection procedure of Leviathan et al. [1] works under temperature sampling. The revised text clarifies how drafted tokens are verified by the target model and how rejected tokens are handled. We also clarify that this procedure preserves the same output distribution as directly sampling from the target model under the same temperature setting.

We also revised Section 3.5 to clarify how this verification procedure is applied to our tree-structured drafts. Specifically, we use tree attention to obtain the target probability distribution of each drafted token in parallel. Following prior work [2, 3], we set the draft distribution of each drafted token as a one-hot distribution, where the drafted token has probability 1 and all other tokens have probability 0. Then, using the above target and draft distributions, we apply the verification/rejection procedure of Leviathan et al. to each path in the draft tree in parallel and select the path with the longest accepted prefix as the final result.

We did not modify the descriptions of Preliminary Prediction and Draft Expansion, because they are draft-stage mechanisms rather than verification-stage mechanisms. They only affect

how candidate draft tokens are constructed before verification, and are orthogonal to the Leviathan et al. verification/rejection procedure. Regardless of whether draft tokens come from Preliminary Prediction, Draft Expansion, or tree construction, the final output is still determined by the same target-model verification procedure.

The revised part in Section 2.1 Line 203 is as follows:

Model-based approach. *Let the target model be denoted as* M_{target}*, and the current input token sequence be denoted as* $X_{1:t}$*. The draft model is denoted as* M_{draft}*. During the draft stage,* M_{draft} *samples* k *draft tokens* $\hat{X}_{t+1:t+k}$ *from the draft distributions:*

$$q_i = M_{\text{draft}}\left(\cdot \mid X_{1:t}, \hat{X}_{t+1:t+i-1}\right), \quad i = 1, \ldots, k.$$

During verification, the drafted tokens are fed into M_{target} *to obtain the target distributions:*

$$p_i = M_{\text{target}}\left(\cdot \mid X_{1:t}, \hat{X}_{t+1:t+i-1}\right), \quad i = 1, \ldots, k.$$

Following Leviathan et al. [23], each drafted token $\hat{x}_{t+i}$ *is accepted with probability* $\min\left(1, \frac{p_i(\hat{x}_{t+i})}{q_i(\hat{x}_{t+i})}\right)$*. If the token is accepted, it is appended to the output and verification continues. Otherwise, verification stops, and a replacement token is sampled from the residual distribution* $\text{norm}(\max(0, p_{t+i} - q_{t+i}))$*.If all* k *drafted tokens are accepted, one additional token is sampled from* $M_{\text{target}}\left(\cdot \mid X_{1:t}, \hat{X}_{t+1:t+k}\right)$*.*

This verification procedure guarantees that the generated tokens follow the same distribution as directly sampling from M_{target} *under the same temperature setting, thereby preserving losslessness.*

The revised part in Section 3.5 Line 609 is as follows:

Finally, the flattened sequence and attention mask are fed into the target model to obtain the target distribution for each drafted token in parallel. Following prior work [11, 28], we treat each draft token as a one-hot draft distribution, where the drafted token has probability 1 and all other tokens have probability 0. We then apply the verification procedure in Section 2.1 to all draft sequences and select the one with the longest accepted prefix as the final result.

[1] Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast inference from transformers via speculative decoding. In International Conference on Machine Learning. PMLR, 19274–19286.

[2] Zhenyu He, Zexuan Zhong, Tianle Cai, Jason Lee, and Di He. 2024. Rest: Retrieval-based speculative decoding. In Proceedings of the 2024 conference of the North American chapter of the association for computational linguistics: Human language technologies (volume 1: long papers). 1582–1595.

[3] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. 2025. Eagle-3: Scaling up inference acceleration of large language models via training-time test. arXiv preprint arXiv:2503.01840 (2025).

[Meta-Review]#2: Sharpening the novelty boundary with a component-level comparison against other retrieval-augmented / hierarchical speculative decoding methods, thereby clarifying where CacheCoder is technically different, rather than mainly an effective incremental combination.

Response: Thank you for the suggestion. We have revised the related work section to sharpen the novelty boundary of CacheCoder through a component-level comparison with recent retrieval-augmented and hierarchical speculative decoding methods.

Specifically, we added discussions of RASD and TriForce in Section 7. We clarify that RASD directly combines retrieved drafts with generated drafts, which may introduce overlap between retrieval and generation and lead to redundant drafting. We also clarify that TriForce builds a hierarchy among multiple generation-based draft models, rather than coordinating retrieval-based reuse with generation-based drafting.

Based on this comparison, we further explain the technical difference of CacheCoder. CacheCoder does not simply place retrieval and generation side by side, nor does it build a hierarchy of multiple generation models. Instead, inspired by cache design in computer architecture, CacheCoder uses retrieved code to guide and reduce draft-model inference, making reuse and generation complementary for better drafting efficiency. These revisions clarify that CacheCoder is not merely an incremental combination of existing techniques, but differs in how it coordinates reuse and generation at the component level.

The revised part in Section 7 Line 1245 is as follows:

Recent studies have further explored retrieval-augmented and hierarchical variants of speculative decoding. RASD [34] augments speculative decoding by directly combining retrieved drafts with generated drafts. However, such a direct combination may introduce overlapping between retrieval and generation and cause redundant drafting. TriForce [39] instead constructs a hierarchy of generation-based models to accelerate generation, where different draft models are organized into multiple generation stages.

CacheCoder differs from these methods by coordinating reuse and generation rather than simply combining them. Inspired by cache design in computer architecture, CacheCoder uses retrieved code to guide and reduce draft-model inference, making reuse and generation complementary for better drafting efficiency.

[Meta-Review]#3: Strengthening the SE-specific significance to avoid reading like a general LLM-inference optimization evaluated only on function-level code benchmarks.

Response: Thank you for this comment. We have added a new discussion in Section 6.4 to strengthen the software-engineering-specific significance of CacheCoder.

The revised text clarifies that code generation naturally involves both reuse and generation: source code contains reusable structures, such as API calls and control-flow patterns [4], while programming tasks often require task-specific logic beyond the retrieved corpus [5].

The revision further relates this observation to how programmers develop software in practice. Rather than writing new functionality entirely from scratch, programmers often start from existing code fragments and adapt them to the current task [6, 7]. Motivated by this practice and by hierarchical cache design in computer architecture, CacheCoder separates the decoding process into two complementary stages: L1 retrieval quickly reuses existing code fragments, and L2 drafting generates additional tokens when retrieval alone is insufficient. This added discussion clarifies that CacheCoder is not only a generic token-level acceleration technique, but a software-engineering-aware decoding strategy that explicitly incorporates code reuse.

The revised part in Section 6.4 Line 1145 is as follows:

Code generation has a software-engineering-specific property: it naturally involves both reuse and generation. Source code contains reusable structures, such as API calls and control-flow pattern [13], while programming tasks often require task-specific logic beyond the retrieved corpus [43].

This property also reflects real-world development practice. Programmers rarely implement new functionality entirely from scratch; instead, they often reuse existing code fragments, and then add task-specific logic to improve development efficiency [7, 37]. Inspired by this practice and the hierarchical cache design in computer architecture, CacheCoder organizes reuse and generation into two complementary stages. L1 retrieval acts like a fast cache to quickly reuse existing code fragments, while L2 drafting generates additional draft tokens when reuse is insufficient.Thus, CacheCoder is not merely a generic token-level acceleration method, but a software-engineering-aware strategy that incorporates code reuse into decoding.

[4] Abram Hindle, Earl T Barr, Mark Gabel, Zhendong Su, and Premkumar Devanbu. 2016. On the naturalness of software. Commun. ACM 59, 5 (2016), 122–131.

[5] Shangwen Wang, Mingyang Geng, Bo Lin, Zhensu Sun, Ming Wen, Yepang Liu, Li Li, Tegawendé F Bissyandé, and Xiaoguang Mao. 2023. Natural language to code: How far are we?. In Proceedings of the 31st ACM joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering. 375–387.

[6] Manuel Sojer and Joachim Henkel. 2010. Code reuse in open source softwaredevelopment: Quantitative evidence, drivers, and impediments. Journal of the Association for Information Systems 11, 12 (2010), 2.

[7] Xiangping Chen, Furen Xu, Yuan Huang, Xiaocong Zhou, and Zibin Zheng.2024. An empirical study of code reuse between GitHub and stack overflow during software development. Journal of Systems and Software 210 (2024), 111964. doi:10.1016/j.jss.2024.111964

[Meta-Review]#4: Incorporating the new rebuttal results into the manuscript to clear the scope and limitations (e.g., sensitivity analyses, ablation study, discussion on FastCoder). In addition, please fix the presentation issues mentioned by reviewers.

4.1 Comment: Sensitivity Analyses

Response: Thank you very much for this comment. We have incorporated the sensitivity analysis from the rebuttal into the manuscript by adding a new research question, RQ4, and a corresponding subsection 5.4, "RQ4: Sensitivity Analysis".

Specifically, we evaluate the impact of three important hyper-parameters: the confidence threshold, the number of expanded nodes, and the retrieval corpus size. The revised results show that the speedup of CacheCoder remains stable when varying the confidence threshold and the number of expanded nodes on CodeLlama-34B under greedy decoding, indicating that CacheCoder does not rely on careful hyper-parameter tuning to achieve consistent acceleration. We also added an analysis of retrieval corpus size, showing that a larger corpus can bring a modest speedup improvement, likely because it provides more relevant retrieved code and improves retrieval hits.

The revised part in Section 5.4, Line 1028, is as follows:

5.4 RQ4: Sensitivity Analysis

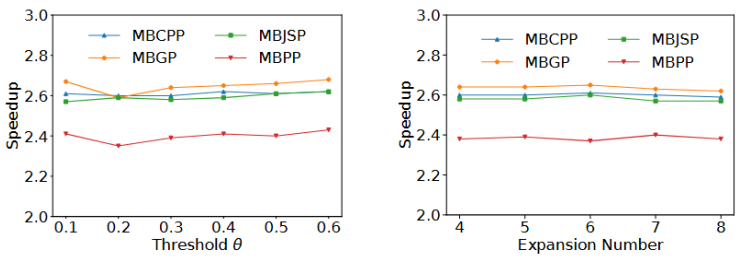


Figure 10: Sensitivity of CacheCoder to the threshold θ and the number of expanded nodes k.

We study the sensitivity of CacheCoder to the confidence threshold θ, the number of expanded nodes k, and the retrieval corpus size. As shown in Figure 10, the speedup of CacheCoder remains stable when varying θ from 0.1 to 0.6 and k from 4 to 8 on CodeLlama-34B under greedy decoding. The results indicate that CacheCoder is robust to different hyper-parameter settings and does not rely on careful tuning to achieve consistent acceleration.

Table 5: Impact of corpus size on CacheCoder with CodeLlama-34B on MBCPP under greedy decoding.

Corpus Size	0.9G	5.0G	15G
Speedup	2.60×	2.67×	2.72×

Table 5 further shows the effect of retrieval corpus size. On MBCPP, increasing the corpus size from 0.9G to 15G improves the speedup from 2.60× to 2.72×. This slight improvement may come from more relevant retrieved code and higher retrieval hit rates.

4.2 Comment: Ablation Study

Response: We appreciate this helpful suggestion. We have incorporated the new ablation results from the rebuttal into Section 5.2 to provide stronger evidence for the key mechanisms of CacheCoder.

Specifically, we added two new ablation variants: "w/o Preliminary Prediction"' and "w/o Draft Expansion". The former removes the preliminary prediction stage and directly extends retrieved code using the draft model, while the latter removes the draft expansion stage and only uses the draft model to filter retrieved code. We also added the corresponding results and analysis in Table 3. The results show that removing either component consistently reduces both speedup and acceptance length. This demonstrates that preliminary prediction is important for filtering low-quality retrieved code before expansion, while draft expansion is necessary for supplementing retrieved code with generated tokens. These additions clarify the contribution of each component and strengthen the ablation study requested in the meta-review.

The revised part in Section 5.2, Line 907, is as follows:

• Without Preliminary Prediction (w/o Pre. Pred.): This variant removes the preliminary prediction stage and directly extends the retrieved code using the draft model. In this setting, retrieved tokens are not pre-filtered before draft-model expansion.

• Without Draft Expansion (w/o Draft Exp.): This variant removes the draft expansion stage. The draft model is only used to filter the retrieved code, without further generating supplementary draft tokens.

The revised part in Section 5.2, Line 955, is as follows:

Impact of Preliminary Prediction and Draft Expansion. *The last two rows of each model show that both preliminary prediction and draft expansion are important to CacheCoder. Removing*

preliminary prediction consistently reduces the speedup and acceptance length, because more low-quality retrieved code is directly passed to draft expansion, making it harder for the draft model to provide additional high-quality draft tokens. Removing draft expansion causes a further drop because the draft model can no longer supplement the retrieved code with generated tokens.

4.3 Comment: Discussion on FastCoder

We appreciate this comment. We have revised Section 4.4 to clarify why FastCoder is discussed separately rather than included as a standard baseline.

Specifically, FastCoder introduces an additional cache of historical generation contexts and verified code. To directly compare with FastCoder, CacheCoder would need to be adapted to use such additional information as well. Therefore, we do not include FastCoder in the main baseline comparison. Instead, we discuss it separately in Section 6.3, where we clarify the relationship and differences between FastCoder and CacheCoder.

The revised part in Section 4.4, Line 694, is as follows:

We do not include FastCoder [50] as a standard baseline because it introduces an additional cache of historical generation contexts and verified code, which provides extra information beyond the baselines considered in our experiments. Therefore, we discuss FastCoder separately in Section 6.3. We also exclude SpecInfer [32] and Medusa [3], as Eagle-3 incorporates their core ideas and achieves stronger performance.

Response to reviewers' comments during rebuttal

[Reviewer#A Q3, Reviewer#B Q1]: Significance of CacheCoder

Response: We have added the revised discussion in Section 6.4, Line 1145, to clarify the software-engineering-specific significance of CacheCoder. The revision explains how the reuse-and-generation nature of code generation motivates our cache-inspired design. The revised part is as follows. For details, please refer to the response to Meta-Review 3.

[Reviewer#A Q1, Reviewer#B Q4]: Decoding Method and Result Explanation

Response: We have added detailed descriptions of the decoding method in Sections 2.1 and 3.5. The revision explains how the verification/rejection procedure works under temperature sampling and how it is applied to our tree-structured drafts. We also explicitly clarify the losslessness of speculative decoding, i.e., the final output distribution is preserved by the target-model verification procedure. For details, please refer to the response to Meta-Review 1.

[Reviewer#A Q4, Reviewer#C Q1] : Impact of Retrieval Corpus Size

Response: We have added experiments on the impact of retrieval corpus size in Section 5.4, Line 1028. For details, please refer to Meta-Review 4.1.

[Review#A Q2]: Comparison with Other Approaches

Response: We have added a comparison between CacheCoder, RASD, and TriForce in Section 7, Line 1245. The revised text clarifies how CacheCoder differs from retrieval-augmented and hierarchical speculative decoding methods by coordinating reuse and generation rather than simply combining them. For details, please refer to the response to Meta-Review 2.

[Review#A Q4]: Extended Ablation Study, Sensitivity Analyses and Definition of ε

Response: We have added the extended ablation study in Section 5.2, Line 907, and the sensitivity analysis in Section 5.4, Line 1028. These revisions incorporate the new experimental results on additional ablation variants and hyper-parameter sensitivity. For details, please refer to Subsections 4.1 and 4.2 in Meta-Review 4.

For the definition of ε, we have revised the caption of Table 6 to make it clearer. The revised caption explicitly states that ε is the maximum remaining suffix length after removing the longest common prefix of the two sequences. The revised part in Section 6.2, Line 1062, is as follows:

Table 6: Ratio of sequences verified by the draft and target models whose gap is within ε, where ε is the maximum remaining suffix length after removing the longest common prefix of two sequences.

Model	$\varepsilon = 0$	$\varepsilon = 1$
Deepseek-Coder-33B	85.12%	94.24%
CodeLlama-34B	86.61%	94.65%

[Review#B Q2]: Explanation for Baseline

Response: We have added an explanation in Section 4.4, Line 694, to clarify the baseline selection. The revised text explains why FastCoder is not directly compared in the main experiments but discussed separately in the Discussion section. It also clarifies that SpecInfer and Medusa are not included because Eagle-3 incorporates their core ideas and achieves stronger performance. For details, please refer to Subsection 4.3 in Meta-Review4.

[Review#B Q3]: Overlap Check

Response: We have added a discussion in Section 6.6 to address the potential data leakage between the evaluation datasets and The Stack used for retrieval. Specifically, we generated

outputs for the validation samples using CodeLlama-34B and computed the 4-gram Jaccard similarity between each generated output and all snippets in the retrieval corpus. For each sample, we used the maximum similarity as its overlap score. The results show that the average similarity is relatively low across all datasets, indicating minimal direct or near-duplicate overlap.

The revised part in Section 6.6, Line 1194, is as follows:

Table 9: Average maximum Jaccard similarity between generated validation outputs and retrieval corpus snippets.

Dataset	MBCPP	MBGP	MBJSP	MBPP
Similarity	0.20	0.16	0.19	0.13

Internal threats. *A potential internal threat is data leakage between the evaluation datasets and The Stack [22] used for retrieval, which could inflate the retrieval hit rate and overestimate the speedup of CacheCoder. To assess this risk, we first generated the output for each validation sample using CodeLlama-34B. For each generated output, we computed its 4-gram Jaccard similarity against every snippet in the retrieval corpus, and used the maximum value as the similarity score of this sample. As shown in Table 9, the average similarity is relatively low across all datasets, indicating minimal direct or near-duplicate overlap.*

[Review#B Q4]: Latency Breakdown

Response: We have added an end-to-end latency breakdown in Section 6.5 to better explain the runtime behavior of CacheCoder. The revised text reports the latency breakdown on CodeLlama-34B over the MBCPP validation set. The results show that the retrieval overhead is negligible compared with model inference. We also clarify that although draft-model inference introduces noticeable latency due to its autoregressive nature, each decoding step can produce multiple tokens, which improves the overall decoding efficiency.

The revised part in Section 6.5, Line 1184, is as follows:

6.5 End-to-End Latency Breakdown

Table 8: End-to-end latency breakdown of CacheCoder

Component	Avg. Time per Decoding Step (s)
Retrieval	0.00088
Draft model inference	0.07029
Target model verification	0.06164

We report the end-to-end latency breakdown of CacheCoder on CodeLlama-34B over the MBCPP validation set in Table 8. The results show that retrieval overhead is negligible compared with model inference. Although draft-model inference incurs noticeable latency due

to its autoregressive nature, each decoding step can produce multiple tokens, thereby improving overall efficiency.

[Review#C Q2]: Impact of Model Scale

Response: We have added a discussion in Section 6.6 to address the potential impact of model scale on CacheCoder. The revised text explains that larger models may contain more redundant layers, making CacheCoder potentially more effective by skipping more layers. In contrast, smaller models may have less redundancy, which could reduce the advantage of self-drafting. We therefore clarify that CacheCoder may be more useful for larger models, while its benefits on smaller models require further investigation.

The revised part in Section 6.6, Line 1207, is as follows:

Prior work [48] shows that larger models tend to contain more redundant layers, making CacheCoder potentially more effective by skipping more layers. In contrast, smaller models such as 7B may have less redundancy, which could reduce the advantage of self-drafting. These suggest that CacheCoder may be useful for larger models, while its benefits on smaller models require further investigation.